

Block Round

The Mathematics of the Project

Technical and didactic documentation of the mathematical foundations employed in the generator of circles, ellipses, spheres and ellipsoids *rendered with Minecraft block textures*. For each concept, the formal definition, the geometric justification, the direct correspondence with the project's source code and the practical translation to in-game construction commands (`/fill`, `/clone`, `//sphere`) are presented.

Topics covered: analytic geometry · discrete rasterization · digital topology · integral calculus · linear algebra · trigonometry · inventory arithmetic · texture mapping · octahedral symmetry.

Table of Contents

1	Overview: mathematical nature of the problem	3
2	Coordinate system and the discrete voxel grid	4
3	The fundamental equation: circle and ellipse	4
4	Euclidean algorithm (distance test)	5
5	Bresenham algorithm (integer midpoint)	6
5.1	Circular case	7
5.2	Elliptic case (two regions)	7
6	Threshold algorithm (corner coverage)	8
7	Rendering modes: Filled, Thin, Thick	9
7.1	Filled	9
7.2	Thin (outline)	9
7.3	Thick	9
8	Discrete area computation	10
9	From 2D to 3D: the ellipsoid equation	10
10	Voxelization and volume computation	11
11	Geometric cuts: planes and the 45° diagonal cut	12
12	Thick 3D mode: tridimensionalization by slices	12
13	3D camera: spherical coordinates	13
14	Auto-zoom: bounding sphere and FOV	13
15	Shading and texture multiplication	14
16	Transition: from continuous mathematics to in-game construction	14
17	Arithmetic of the Minecraft inventory	16
17.1	Single slot, single stack, full pack	16
17.2	Storage: chests, double chests, shulker boxes	16
17.3	Reference table: canonical spheres	16

18 Highlight overlay and exposed-face count	17
18.1 What counts as an exposed face	17
18.2 Boundary detection formula	17
18.3 Exposed-face count as a survival heuristic	17
19 Layer-by-layer construction (horizontal decomposition)	18
20 Optimization via octahedral symmetry (O_h)	19
20.1 Concrete command sequence	19
20.2 Time-cost comparison	19
21 Block textures and visual composition	20
21.1 The six faces and UV mapping	20
21.2 Block families and tones	20
21.3 Gradients by mixing blocks	21
22 Conclusion	21
Appendix A: Construction reference tables and worked example	23

1. Overview: mathematical nature of the problem

Block Round is a generator of rounded shapes *textured with Minecraft blocks*: circles and ellipses in 2D, spheres and ellipsoids in 3D. The central problem belongs to the class of *discrete rasterization*: given a curve or surface defined analytically over the real numbers $\mathbb{R}$, determine which cells of a regular grid (pixels in 2D, voxels in 3D) must be marked to represent it. In Block Round each marked cell additionally receives a six-face texture from the Minecraft palette (Grass Block, Cobblestone, Oak Planks, Quartz, Diamond, etc.), so the resulting figure is not just a mathematical silhouette but a *buildable structure* the user can re-create in survival mode, in creative mode with `/fill`, or import via Sponge Schematic v2 (`.schem`) into WorldEdit, Litematica or MCEdit.

This problem admits no unique solution. Different inclusion criteria produce different discrete silhouettes, and each criterion has its own mathematical justification. For this reason the project implements three distinct 2D algorithms (Euclidean, Bresenham, Threshold), three rendering modes (Filled, Thin, Thick) and three 3D shading styles, all described in the subsequent sections. The choice between algorithms is decided not just by visual smoothness but also by the number of blocks the user will need to gather — a sphere of diameter $D = 20$ requires 4 189 blocks under the Euclidean algorithm but as many as 4 322 blocks under Threshold.

Execution occurs entirely in the browser, with no server component, which imposes on the mathematical formulation an additional requirement of *computational efficiency* (a Diamond sphere of $D = 32$ must render at 60 fps including 17 156 textured voxels). The theoretical areas involved include:

- analytic geometry of conics and quadrics;
- incremental algorithms with integer arithmetic (Bresenham);
- digital topology (4-, 6- and 26-connected neighborhoods);
- integral calculus and counting measure;
- linear algebra (cutting planes, perspective projection);
- trigonometry and spherical coordinates;
- combinatorial inventory arithmetic and group symmetry.

What is new compared to its sibling Pixel Round. Block Round adds five new mathematical concerns that arise only when the rendered cells become *Minecraft blocks*: (i) the UV mapping of six faces per voxel; (ii) the arithmetic of the survival inventory (packs of 64 items, single chests of 27 slots, double

chests of 54 slots); (iii) the highlight overlay used to count exposed faces during construction; (iv) the decomposition of a 3D shape into horizontal layers, since real-world building proceeds floor by floor; (v) the exploitation of the octahedral symmetry group O_h to reduce the manual block-placement cost by a factor of 8 via `/clone`.

2. Coordinate system and the discrete voxel grid

The canvas is a grid of $G_x \times G_y$ cells in 2D, extended to $G_x \times G_y \times G_z$ voxels in 3D. Each cell (i, j) occupies the unit square $[i, i+1] \times [j, j+1]$; in 3D, each voxel (i, j, k) occupies the unit cube $[i, i+1] \times [j, j+1] \times [k, k+1]$. In continuous code, the **center** of the cell is at $(i + \frac{1}{2}, j + \frac{1}{2})$. This convention is critical: comparing the circumference to the *corner* (i, j) or to the *center* $(i + \frac{1}{2}, j + \frac{1}{2})$ changes which cells are considered as belonging to the shape, and therefore which Minecraft blocks the user must place.

$$\text{center of cell } (i, j) = (i + \frac{1}{2}, j + \frac{1}{2})$$

For a shape of diameter D , the code expands the grid to $D+2$ cells along each axis (a margin of one cell on each side). This ensures that extreme pixels (N, S, E, W) never fall on the matrix boundary. The **center** of the shape is at $(G_x/2, G_y/2)$, and the *semi-axes* are $r_x = D/2$ and $r_y = D/2$ (or $W/2$ and $H/2$ for an ellipse). In 3D the same logic extends to a margin along the z axis. The unit cell of the canvas is the same as the unit block of Minecraft, so coordinates in the algorithm map one-to-one to in-game (x, y, z) coordinates — a sphere of $r = 8$ centred at the canvas centre maps to a sphere of $r = 8$ centred at any world position the user selects.

Minecraft mapping. If the user spawns at $(0, 64, 0)$ and wants a sphere of $D = 16$ ($r = 8$) sitting on the ground, the centre in world coordinates should be placed at $(0, 72, 0)$ (i.e. 8 blocks above the surface) so that the south pole of the sphere touches the floor exactly. The corresponding WorldEdit command is `//sphere stone 8` executed while standing at $(0, 72, 0)$.

3. The fundamental equation: circle and ellipse

All theory in the project begins with the implicit equation of the ellipse centered at the origin with semi-axes r_x and r_y :

$$\left(\frac{x}{r_x}\right)^2 + \left(\frac{y}{r_y}\right)^2 = 1$$

When $r_x = r_y = r$, the ellipse degenerates into a **circumference** of radius r . Points in the *interior* satisfy ≤ 1 ; points in the *exterior*, > 1 . This is the inequality that determines whether a voxel belongs to the shape and therefore whether the user places a block there. The three algorithms are, fundamentally, three different ways of applying (or approximating) this test on a grid.

Shifting the center to (c_x, c_y) and evaluating at the center of each cell:

$$d_x = \frac{i + \frac{1}{2} - c_x}{r_x}, \quad d_y = \frac{j + \frac{1}{2} - c_y}{r_y}, \quad \text{inside} \iff d_x^2 + d_y^2 \leq 1$$

Minecraft mapping. For a tower in survival, the user typically wants a *wide flat circle* for the foundation and progressively smaller circles as the tower rises. Each storey is one application of the implicit ellipse equation with the same semi-axes but a different z . Block Round computes the 2D layer once and the user simply duplicates that layer with `/clone x1 y1 z1 x2 y2 z2 x3 y2+1 z3` per storey.

4. Euclidean algorithm (distance test)

Idea. For each row j , analytically find the columns i that lie inside the ellipse. Solving the ellipse equation for x , given y :

$$x = c_x \pm r_x \sqrt{1 - \left(\frac{y - c_y}{r_y}\right)^2}$$

For a row j whose vertical displacement is $d_y = j + \frac{1}{2} - c_y$, the *horizontal half-width* of the ellipse at that height is:

$$dxM = r_x \sqrt{1 - \frac{d_y^2}{r_y^2}}$$

If $d_y^2/r_y^2 \geq 1$, the row does not intersect the ellipse and is skipped. Otherwise, every column i such that $c_x - dxM \leq i + \frac{1}{2} \leq c_x + dxM$ is filled. The integer bounds come from:

$$lo = \lceil c_x - dxM - \frac{1}{2} \rceil, \quad hi = \lfloor c_x + dxM - \frac{1}{2} \rfloor$$

Justification. The $+\frac{1}{2}$ term arises from the convention that cell i has center at $i + \frac{1}{2}$. The functions $\lceil \cdot \rceil$ and $\lfloor \cdot \rfloor$ perform the conversion from a continuous interval to integer indices, ensuring that only cells whose **center** lies in the interior of the ellipse are included. This formulation yields the discrete approximation closest to the continuous curve and, consequently, the silhouette of greatest visual smoothness. For small diameters, however, the result may exhibit a less pronounced pixel-art character compared to the other algorithms — a $D = 5$ Euclidean circle has only 13 blocks, while the Threshold variant of the same diameter has 21.

Minecraft mapping. The Euclidean criterion is the one closest to what `//sphere` produces in WorldEdit. To build a $D = 16$ ($r = 8$) Cobblestone sphere using the Euclidean shell only, the volume is $V \approx \frac{4}{3}\pi(8)^3 \approx 2145$ blocks under filled mode, dropping to roughly 804 blocks under thin mode (the shell) — that is the difference between gathering 34 Cobblestone packs ($34 \times 64 = 2176$) and 13 packs ($13 \times 64 = 832$).

5. Bresenham algorithm (integer midpoint)

The Bresenham algorithm, published in 1965 for drawing line segments, was later extended to circles and generalized by Pitteway and Kappel to arbitrary ellipses. Its distinguishing property is to dispense with floating-point operations and square root extraction: determining the next pixel uses exclusively **integer additions and comparisons**, by means of a *decision function* that evaluates on which side of the analytic curve the midpoint between the two candidate pixels lies. For Minecraft building this is the algorithm that gives the most *recognisable* pixel-art look — the same stepped silhouette that the community has been hand-building since 2011.

5.1 Circular case

For a circle of integer radius R , the algorithm starts at $(x=0, y=R)$ with the initial decision parameter:

$$d_0 = 1 - R$$

At each step (in the octant from 0 to 45°):

$$\begin{cases} \text{if } d < 0 : & \text{next is } (x+1, y), \quad d += 2x + 3 \\ \text{if } d \geq 0 : & \text{next is } (x+1, y-1), \quad d += 2(x - y) + 5 \end{cases}$$

Justification. The function d approximates, at each iteration, the value of the implicit circle equation evaluated at the **midpoint** between the two candidate pixel options:

$$F(x+1, y - \tfrac{1}{2}) = (x+1)^2 + (y - \tfrac{1}{2})^2 - R^2$$

The sign of d indicates whether the midpoint lies inside the circle (in which case one advances only horizontally) or outside (in which case the vertical coordinate is also decremented). The eight octants of the circle are obtained by applying the symmetries of the dihedral group D_4 to the computed octant, which corresponds, in code, to the eight `span(...)` calls. The resulting trace exhibits the stepped pattern characteristic of discrete approximations of smooth curves — and is precisely the silhouette that classical Minecraft-pixel-art tutorials describe.

5.2 Elliptic case (two regions)

For an ellipse with semi-axes a, b , the algorithm divides the first quadrant into **two regions**, delimited by the point where the tangent makes 45°:

$$\text{Region I: } 2b^2x < 2a^2y \quad (|dy/dx| < 1), \quad \text{Region II: otherwise}$$

The initial decision parameters are:

$$\begin{aligned} p_1 &= b^2 - a^2b + \frac{a^2}{4}, \\ p_2 &= b^2(x + \tfrac{1}{2})^2 + a^2(y - 1)^2 - a^2b^2. \end{aligned}$$

At each iteration, p is updated by precomputed increments (all integers, after multiplying by 4). The result is the minimum possible arithmetic per pixel: *one sign test and two additions*. The code uses `Math.floor` when computing a and b from r_x and r_y to avoid adding an extra row outside the bounding box when r_y is non-integer.

Minecraft mapping. For survival builders the Bresenham silhouette is often preferable to Euclidean because its discrete steps are easier to count manually — the user knows that the next block is always either *straight* or *diagonal*, which keeps the arm-position consistent while placing. A $D = 11$ Bresenham circle has the canonical 80-block sequence published in dozens of community guides.

6. Threshold algorithm (corner coverage)

This algorithm asks: *is there any corner of the cell inside the ellipse?* For each cell (i, j) , the code finds the corner closest to the center of the shape (projection of the center onto the cell edges):

$$n_x = \text{clamp}(c_x, i, i+1), \quad n_y = \text{clamp}(c_y, j, j+1)$$

$$\text{inside} \iff \left(\frac{n_x - c_x}{r_x} \right)^2 + \left(\frac{n_y - c_y}{r_y} \right)^2 \leq 0,94$$

The value **0,94** is an *empirical threshold* slightly below 1,0. Its purpose is to eliminate the artifact known as the *tangential spike* of 1 pixel that occurs at the four cardinal points (N, S, E, W), where the curve is tangent to a full edge of the cell. Without this reduction, the algorithm would include an additional cell whose contribution does not improve the visual representation of the shape.

Among the three algorithms, this is the one that produces the silhouette of largest area for the same diameter D , since the inclusion condition is the least restrictive: a single corner of the cell satisfying the ellipse inequality suffices to fill the entire cell. For Minecraft, this is the algorithm that gives the most *rounded* look but at the cost of placing more blocks — $D = 16$ produces 213 shell blocks under Threshold versus 200 under Bresenham and 192 under Euclidean.

Minecraft mapping. Threshold is the algorithm of choice when the build is meant to be seen from far away (e.g. a Quartz dome on top of a temple), because the slightly larger silhouette reads more clearly through ambient occlusion shadows. The trade-off is roughly 5–10% more blocks per shell relative to Euclidean.

7. Rendering modes: Filled, Thin, Thick

Having a matrix $f[j][i] = 1$ for interior cells, the project derives three visual styles by **discrete topology**:

7.1 Filled

Returns f unchanged. All interior cells are blocks. This is what `//sphere stone 8` produces.

7.2 Thin (outline)

A cell belongs to the thin contour if it is filled *and* has at least one orthogonal neighbor (N, S, E, W) *outside* the shape. In logical notation:

$$b(i, j) = f(i, j) \wedge (\neg f(i-1, j) \vee \neg f(i+1, j) \vee \neg f(i, j-1) \vee \neg f(i, j+1))$$

Geometrically: this is the **discrete boundary** of the shape, the discrete analog of the topological boundary $\partial F = F \cap \overline{F^c}$. In Minecraft this is the equivalent of `//hsphere` (hollow sphere): only the shell is materialised.

7.3 Thick

The thick mode adds to the boundary cells the **diagonal bridges**: interior cells that have simultaneously a horizontal and a vertical boundary neighbor. This fills the *empty diagonal cells* that thin mode leaves between the shell blocks, useful when the contour must be a continuous surface with no openings — particularly important for Glass and Ice domes, where each empty diagonal cell of the shell is, in Minecraft, an air block through which water, mobs and light propagate into the interior.

$$\begin{aligned} t(i, j) &= b(i, j) \vee (f(i, j) \wedge h_H \wedge h_V), \\ h_H &= b(i-1, j) \vee b(i+1, j), \\ h_V &= b(i, j-1) \vee b(i, j+1). \end{aligned}$$

Minecraft mapping. For a $D = 20$ Glass dome (hollow), the thick mode yields a *continuous* shell of 608 blocks against ≈ 510 blocks for the thin mode — the ~ 100 extra blocks are diagonal-bridge stitching that fills the empty diagonal cells left by voxelisation, closing the interior to water, mobs and light propagation.

8. Discrete area computation

The `area2D` function simply **counts** the cells marked in f . This is the *counting measure*, which approximates the Riemann integral of the indicator function of the ellipse:

$$\underbrace{\pi r_x r_y}_{\text{continuous area}} \quad \longleftrightarrow \quad \underbrace{\sum_{j=0}^{G_y-1} \sum_{i=0}^{G_x-1} f(i, j)}_{\text{discrete area}}$$

For large D , discrete area/continuous area $\rightarrow 1$. For small D , the difference between algorithms is visible: Threshold tends to overestimate; Euclidean stays close to the ideal area; Bresenham falls somewhere between the two.

Minecraft mapping. The block-count chip in the Block Round UI displays exactly this number: 268 for $D = 8$, 904 for $D = 12$, 2145 for $D = 16$. The chip then expresses the count as N (*stacks* $\times 64$ + *remainder*) — so $2145 = 33 \times 64 + 33$, i.e. “33 stacks plus 33 items”, which is what the user actually has to gather.

9. From 2D to 3D: the ellipsoid equation

In 3D, the fundamental shape is the **ellipsoid** centered at (c_x, c_y, c_z) with semi-axes r_x, r_y, r_z . Its equation is the natural generalization of the ellipse:

$$\left(\frac{x}{r_x}\right)^2 + \left(\frac{y}{r_y}\right)^2 + \left(\frac{z}{r_z}\right)^2 = 1$$

When $r_x = r_y = r_z$, we recover the **sphere**. Evaluating at the center of each voxel:

$$a_\xi = \frac{\xi + \frac{1}{2} - c_\xi}{r_\xi} \quad (\xi \in \{x, y, z\}), \quad \text{inside} \iff a_x^2 + a_y^2 + a_z^2 \leq 1$$

Minecraft mapping. An ellipsoid with $W = 20$, $H = 10$, $D = 20$ produces a *flattened sphere* ideal for an underground dome or a stadium roof. The volume is $V = \frac{4}{3}\pi \cdot 10 \cdot 5 \cdot 10 \approx 2094$ blocks, equivalent to 33 stacks of any block type. Equivalent WorldEdit command: `//ellipsoid stone 10 5 10`.

10. Voxelization and volume computation

The continuous volume of the ellipsoid is a classical result of integral calculus, obtained by integration over disks:

$$V = \frac{4}{3} \pi r_x r_y r_z$$

The discrete version iterates over every voxel of the box $D_x \times D_y \times D_z$ and increments a counter when $a_x^2 + a_y^2 + a_z^2 \leq 1$. This is the `voxelVolume` algorithm.

Shell vs. interior. For the *filled* mode, the project displays only voxels with at least one 6-neighbor outside the preserved set (that is, voxels visible from the external surface or from the cut face). For *thin*, only the **surface** of the complete ellipsoid is shown (1-voxel thickness). In a Minecraft survival run only the visible faces need to receive their dedicated textured block — the interior voxels can be left as filler (e.g. Dirt instead of Diamond) to save resources.

Reference table. The table below summarises the canonical sphere diameters most commonly requested in Minecraft community builds, translated into number of blocks (V), packs of 64 ($P = \lceil V/64 \rceil$) and double chests of $54 \times 64 = 3456$ slots ($DC = \lceil V/3,456 \rceil$). Use this table to plan resource gathering before construction.

Diameter	Blocks (V)	Packs ($\times 64$)	Double chests	Note
D=8	268	5	1	small dome
D=12	904	15	1	medium sphere
D=16	2145	34	1	big tower top
D=20	4189	66	2	stadium roof
D=24	7238	114	3	large dome
D=32	17156	269	5	mega-build

Filled-mode volumes computed under the Euclidean algorithm at $D \in \{8, 12, 16, 20, 24, 32\}$. Thin-mode (shell) values are roughly $V_{\text{shell}} \approx 4\pi r^2/1$ block-thick, i.e. between 25% and 40% of the filled volume depending on D .

11. Geometric cuts: planes and the 45° diagonal cut

Block Round allows cutting the solid by **three planes**. Each cut is a *linear inequality* that discards voxels:

$$\begin{aligned} \text{X axis:} \quad & x \geq \textit{cut} \quad \Rightarrow \text{discarded} \\ \text{Y axis:} \quad & y \geq \textit{cut} \quad \Rightarrow \text{discarded} \\ \text{Diagonal:} \quad & x + y \geq \textit{cut} \quad \Rightarrow \text{discarded} \end{aligned}$$

The X and Y planes are perpendicular to the coordinate axes, with normal vectors $(1, 0, 0)$ and $(0, 1, 0)$. The *diagonal plane* has normal vector $(1, 1, 0)/\sqrt{2}$ and cuts at 45° in the *XY* plane: mathematically, this is the family of planes $x + y = k$. Since the maximum of $x + y$ in a box $D_x \times D_y$ is $D_x + D_y$, the diagonal cut slider has range $D_x + D_y$, while X and Y have ranges D_x and D_y , respectively. The cut is **proportional**: the code stores a percentage *cutPct* and recomputes the limit as

$$\textit{cut} = \textit{cutPct} \cdot \max_{\text{axis}}$$

so 50% on one axis remains 50% when the user switches axis or resizes.

Minecraft mapping. The Y-axis cut at 50% on a sphere produces the canonical *dome* of half the volume. For $D = 16$: $V_{\text{full}} = 2\,145$ blocks, $V_{\text{dome}} \approx 1\,095$ blocks (≈ 18 packs of 64). The diagonal cut at 50% produces a *half-bowl* cross-section frequently used for fountain interiors and amphitheatre seating.

12. Thick 3D mode: tridimensionalization by slices

For *thick* mode in 3D, the project applies the 2D thick algorithm to **all slices** along the three axes: *XY* for each z , *XZ* for each y , *YZ* for each x . It then takes the *union* of the results. The geometric motivation is closing the shell: the minimum set of voxels that fill every empty diagonal cell without doubling the shell thickness. Formally, with $S_z(z)$ the *XY* slice at height z and T the 2D thick operator:

$$\textit{thick3D} = \bigcup_z T(S_z(z)) \cup \bigcup_y T(S_y(y)) \cup \bigcup_x T(S_x(x))$$

The result is then intersected with the set preserved by the cut. The underlying theory is **digital topology**: the thick operator guarantees *26-connectivity* of the shell (neighbors including diagonals). This matters in Minecraft because even when two shell voxels are diagonal neighbours (sharing only an edge), the cell between them remains an air block that water and creatures cross into the interior — thick mode closes those paths.

Minecraft mapping. For an underwater Glass dome on a survival ocean monument, thick mode is mandatory: the diagonal bridges fill the empty cells left by thin mode, eliminating the air-block paths through which water would otherwise invade. Cost: $D = 20$ dome at thick mode requires ≈ 380 Glass blocks vs. ≈ 320 under thin mode — 19% more material in exchange for a completely closed shell.

13. 3D camera: spherical coordinates

The 3D camera orbits the object at a distance r with two angles — θ (azimuth, in the horizontal plane) and φ (polar, from the vertical axis). This is the **physical convention** of spherical coordinates, and the conversion to Cartesian (the form `three.js` expects) is:

$$\begin{aligned}x &= r \sin(\varphi) \cos(\theta), \\y &= r \cos(\varphi), \\z &= r \sin(\varphi) \sin(\theta).\end{aligned}$$

Initial position: $\theta = \pi/4$ (45° , isometric perspective) and $\varphi = \pi/3$ (60° from the north pole, slightly above the equator). Mouse drag increments θ and φ directly; wheel/pinch increments r . The simplicity of this parametrization is what allows free rotation without quaternions or explicit matrices.

14. Auto-zoom: bounding sphere and FOV

When the user changes the figure size or resets the camera, the project recomputes the **ideal distance** of the camera so that the object fits the viewport at any angle. The idea is to enclose the object in a *sphere of radius R* (half the diagonal of the AABB, axis-aligned bounding box):

$$R = \sqrt{(D_x/2)^2 + (D_y/2)^2 + (D_z/2)^2}$$

A sphere has **projection invariant under rotation**, so if it fits in the viewport at one orientation, it fits in all. Given the vertical FOV of the camera ($fov_V = 35^\circ$ in radians), the distance required to frame a sphere of radius R is:

$$dist_V = \frac{R}{\tan(fov_V/2)}$$

The horizontal FOV follows from the canvas aspect ratio ($aspect = \text{width}/\text{height}$) by:

$$fov_H = 2 \arctan(\tan(fov_V/2) \cdot aspect)$$

The final distance is the maximum of $dist_V$ and $dist_H$, multiplied by 1,20 (20% visual margin). When the figure is cut, the code shrinks D_x or D_y to the remaining size, so that a sphere cut at 75% does not appear tiny in a corner. When the oak-tree easter egg or the creeper easter egg is active, the bounding box is extended upward to include the entity height.

15. Shading and texture multiplication

The three 3D styles (*Classic*, *Smooth*, *Blocks*) differ by **multiplicative factors** applied to the three visible faces of each voxel (top, lit side, dark side). The `shadeHex(hex, factor)` function parses the hex into (R, G, B) and multiplies each channel, with clamping to $[0, 255]$ — and the result becomes a tint applied on top of the raw block texture:

$$R' = \text{clamp}(R \cdot f), \quad G' = \text{clamp}(G \cdot f), \quad B' = \text{clamp}(B \cdot f)$$

This is a *linear approximation* of the ideal Lambertian illumination function,

$$I = \max(0, \mathbf{n} \cdot \mathbf{l}) \cdot \text{texture},$$

where the three factors correspond to the cosine of the angle between the normal of each face and a fixed light direction. In *Smooth* the factors are close (similar faces); in *Classic* the factors differ more (strong contrast, the look of vanilla Minecraft). *Blocks* additionally introduces a geometric inset — a constant translation of each face inward to reveal the inter-voxel boundaries even when adjacent voxels carry the same texture. For emissive blocks (Glowstone, Sea Lantern, Shroomlight, Magma, Crying Obsidian) the Lambertian term is replaced by a constant $I = \text{emissive_factor} \cdot \text{texture}$ scaled to the block's MC light level.

16. Transition: from continuous mathematics to in-game construction

The fifteen sections above describe the *continuous-to-discrete* pipeline that Block Round shares with its sibling project Pixel Round: from the implicit equation of the ellipse all the way to the camera that frames the result. The remaining six sections describe what is **specific** to Block Round — the additional layer of mathematics introduced by the fact that the rendered cells are not abstract pixels but *Minecraft blocks*, each with six textured faces, a weight in the inventory and a cost in playtime to gather and to place.

These are not cosmetic considerations. The decision to render with block textures changes the operational problem the user faces: instead of *exporting a PNG*,

the user must *translate the silhouette into a placement plan*. The mathematical content of the next sections is concerned exactly with that translation: inventory arithmetic, face-count optimization, layer-by-layer decomposition, and the exploitation of symmetry to reduce the cost from N block placements to $N/8 + 7$ command invocations.

17. Arithmetic of the Minecraft inventory

A Minecraft player carries items in a 36-slot *inventory*, organised as a 9×4 grid. Each slot holds up to 64 items of the same type (a *stack* or *pack*). The player can also access *storage*: a single chest is 27 slots, a double chest is 54 slots. The arithmetic that translates a target block count into a gathering plan is therefore a *ceiling-division* problem.

17.1 Single slot, single stack, full pack

The *stack* is the elementary unit of inventory accounting. Each stack contains at most 64 items. The conversion from block count N to packs (or stacks) is

17.2 Storage: chests, double chests, shulker boxes

To compute how many *double chests* a build requires, divide the block count by 54 slots $\times$ 64 items/slot = 3 456 items per double chest:

$$\text{packs : } N_{\text{packs}} = \left\lceil \frac{N}{64} \right\rceil \quad \text{double chests : } N_{\text{dc}} = \left\lceil \frac{N}{3456} \right\rceil$$

A *single chest* holds $27 \times 64 = 1\,728$ items. A *shulker box* (portable container) holds $27 \times 64 = 1\,728$ items but can itself be stored *inside* a chest slot, so a double chest filled with 54 shulker boxes can hold up to $54 \times 1\,728 = 93\,312$ items. For mega-builds the relevant capacity unit is the *shulker-loaded double chest*.

17.3 Reference table: canonical spheres

The table below converts the canonical filled-sphere volumes into gathering plans. The last column suggests a build context for each diameter:

Diameter	V (blocks)	Packs	Storage	Typical build
D=8	268	5	1 dc	hut roof, well dome
D=12	904	15	1 dc	watchtower top
D=16	2145	34	1 dc	villager bell, library dome
D=20	4189	66	2 dcs	observatory dome
D=24	7238	114	3 dcs	temple dome
D=32	17156	269	5 dcs	cathedral dome, world boss arena

The *stacks-and-remainder* representation $N = q \cdot 64 + r$ shown by Block Round’s info chip mirrors the way the survival inventory displays partial stacks: the player

sees q full slots plus one slot showing r . For $D = 16$, the chip prints “2 145 (33 × 64 + 33)”, telling the player to fill 33 slots completely plus one slot with 33 items — exactly 34 slots, which fits in a single inventory row plus four extra slots.

18. Highlight overlay and exposed-face count

Block Round renders, by default, a black *highlight overlay* on the edges of every visible voxel face. Visually this matches the “F3+G” chunk-border style of the in-game debug overlay, scaled down to the per-voxel level. Mathematically the overlay implements the visualization of the **exposed-face count**: a number that survival builders use to validate progress block-by-block.

18.1 What counts as an exposed face

A face of voxel $\mathbf{v}$ shared with neighbor $\mathbf{n}$ is *exposed* if and only if $\mathbf{n} \notin V_{\text{solid}}$ (the neighbor is empty). Each solid voxel contributes between 0 (fully buried, internal to the solid) and 6 (isolated) faces. The total exposed-face count F_{exp} is the discrete analog of the *surface area* of the figure.

18.2 Boundary detection formula

A voxel is on the boundary if at least one of its six face-neighbors is outside the solid:

$$b(i, j, k) = f(i, j, k) \wedge (\neg f(i \pm 1, j, k) \vee \neg f(i, j \pm 1, k) \vee \neg f(i, j, k \pm 1))$$

This is the same boundary operator as the 2D thin algorithm extended to 3D, and the highlight overlay simply draws the cube-edge lines of every boundary voxel. For Glass and Ice the default is OFF, since alpha-blended frames already do the job; for Cobblestone, Stone, Diamond and other opaque blocks the default is ON, because the overlay disambiguates internal voxels from the shell.

18.3 Exposed-face count as a survival heuristic

For survival builders the relevant count is not the total volume V but the *exposed-face count* F_{exp} , because that is what is visible from the outside. The total F_{exp} over all solid voxels equals six times the boundary voxel count minus twice the number of face-adjacencies among boundary voxels:

$$F_{\text{exp}} = \sum_{\mathbf{v} \in V_{\text{solid}}} \sum_{\mathbf{n} \in N_6(\mathbf{v})} [\mathbf{v} + \mathbf{n} \notin V_{\text{solid}}]$$

For a $D = 16$ sphere (filled mode), $V = 2145$, $V_{\text{boundary}} \approx 432$ (the shell), and $F_{\text{exp}} \approx 1184$ visible faces — so the build needs only about 432 *visible* blocks; the remaining 1713 blocks are interior and can be filled with the cheapest block available (Dirt, Cobblestone). The highlight overlay lets the builder spot a missing block in the shell immediately, because a missing voxel produces a discontinuity in the overlay pattern that the eye picks up at a glance.

19. Layer-by-layer construction (horizontal decomposition)

In Minecraft, construction proceeds *by horizontal layers*: the player stands on a layer, places the blocks of the layer above, then steps up one block (jumps + places a temporary scaffolding block, or uses an elytra). The 3D ellipsoid problem is therefore best understood as a *stack of 2D ellipses*, one per layer:

$$\text{layer}(k) = \{ (i, j) : a_x(i)^2 + a_y(j)^2 \leq 1 - a_z(k)^2 \}$$

For each integer layer k from $-r_z$ to $+r_z$, the cross-section is itself an ellipse with shrunk semi-axes $r_x(k)$ and $r_y(k)$:

$$r_x(k) = r_x \sqrt{1 - \left(\frac{k}{r_z}\right)^2}, \quad r_y(k) = r_y \sqrt{1 - \left(\frac{k}{r_z}\right)^2}$$

This means the 3D problem is reduced to *computing the 2D algorithm of the previous sections, one layer at a time*, and stacking the results. Cognitively this is a massive simplification: instead of mentally tracking three coordinates the builder tracks two coordinates per layer plus the layer index. Each layer becomes a 2D problem already solved by the algorithms of Sections 4–6.

Minecraft mapping. For a sphere of $D = 16$ ($r_z = 8$), the equatorial layer ($k = 0$) is the full $D = 16$ Bresenham circle (≈ 200 blocks). The next layer up ($k = 1$) is a $D = 15,97$ circle, effectively the same shape minus the corner cells. By $k = 4$ the layer is a $D = 13,86$ circle (about 148 blocks), and by $k = 7$ it has shrunk to a $D = 7,48$ circle (about 43 blocks). The poles ($k = \pm 8$) are a single-block cap.

The Block Round info chip exposes this layer count in 3D mode when the user activates the Center-guides corner button (key c). The horizontal slices are drawn as a translucent yellow cross at the equator and at each $r_z/2$ subdivision, giving the builder a clean reference for where the layer boundaries fall.

20. Optimization via octahedral symmetry (O_h)

A sphere centred at the origin is invariant under the action of the **octahedral group** O_h of order 48. The voxelization of the sphere preserves a subgroup of order 48 generated by the three coordinate reflections and the three $\pi/2$ rotations around each axis. For practical building purposes the relevant subgroup is the order-8 *octant* reflection group $\mathbb{Z}_2^3$, generated by $x \mapsto -x$, $y \mapsto -y$, $z \mapsto -z$. The voxel set in one octant determines the full sphere:

$$V_{\text{total}} \approx 8 V_{\text{octant}} \implies \text{reduction} = \frac{N - N/8}{N} = 87,5\%$$

This is the largest mathematical optimisation available to a Minecraft builder. Instead of manually placing N blocks, the builder places $N/8$ blocks in the positive octant and then issues seven `/clone` (vanilla) or `//copy + //paste` (WorldEdit) commands with appropriate reflections.

20.1 Concrete command sequence

For a sphere of $D = 24$ ($r = 12$, $V = 7\,238$ blocks) centred at world coordinates $(0, 72, 0)$, build the positive octant first (the $+x, +y, +z$ region, ≈ 905 blocks), then issue:

```
/clone 0 72 0 12 84 12 ~~~ filtered minecraft:stone
/clone 0 72 0 12 84 12 -12 72 0 mode:masked (-x reflection)
/clone 0 72 0 12 84 12 0 60 0 mode:masked (-y reflection)
/clone 0 72 0 12 84 12 0 72 -12 mode:masked (-z reflection)
/clone -12 72 0 -1 84 12 mode:masked (-x, -y octant)
/clone 0 60 -12 12 71 -1 mode:masked (-y, -z octant)
/clone -12 72 -12 -1 84 -1 mode:masked (-x, -z octant)
/clone -12 60 -12 -1 71 -1 mode:masked (-x, -y, -z octant)
```

Each `/clone` costs about 50 ms of server time, so the seven reflections complete in less than half a second. The manual placement of the 905 octant blocks takes about 15 min in survival or 45 s in creative with a brush. The full 7 238-block placement (without symmetry) would take ≈ 2 h in survival or 6 min in creative.

20.2 Time-cost comparison

Let T_{block} be the per-block placement time (typically 1–2 s in survival, 0,1–0,5 s in creative) and T_{clone} the per-clone command time (≈ 50 ms). The total construction

time with and without symmetry is:

$$T_{\text{octant}} + 7 T_{\text{clone}} \ll T_{\text{full}}$$

For $N = 7\,238$ in survival ($T_{\text{block}} = 2\text{ s}$, $T_{\text{clone}} = 0,05\text{ s}$): full = $14\,476\text{ s} \approx 4\text{ h } 1\text{ min}$; octant + clones = $1\,810\text{ s} + 0,35\text{ s} \approx 30\text{ min}$. A factor-8 speed-up that closely matches the order of the reflection subgroup. The full octahedral group O_h of order 48 could in principle reduce further (by a factor of 48), but exploiting the diagonal rotations requires a 45° -rotated build area, which is rarely worth the setup cost in vanilla Minecraft.

21. Block textures and visual composition

A Minecraft block is not a flat-coloured cell but a *cube with six textured faces*. Block Round renders each voxel with its six canonical face textures, sampled from the vanilla resource pack. Mathematically the texturing operation is a *UV mapping*: each face of the unit cube is parameterised by $[0, 1]^2$ coordinates, and the corresponding texture is sampled at those UVs to produce the rendered pixels.

21.1 The six faces and UV mapping

For each voxel face the renderer consults the block model:

$$\text{face} \in \{\text{top, side, bottom}\} \quad \text{UV} : [0, 1]^2 \rightarrow \text{pixels}$$

Most blocks (Stone, Cobblestone, Diamond Block, Oak Planks) use the *same texture on all six faces*. Multi-face blocks (Grass Block, Pumpkin, Hay Bale, Quartz Pillar, all wood Logs, Crafting Table, Furnace, Bookshelf, TNT) use *different textures* on top, sides and bottom: Grass Block has a green grass top, a grass-and-dirt side and a plain dirt bottom; an Oak Log has the wood-grain end on top and bottom and bark on the four sides. Crucially the *mathematics of which voxels to place* does not depend on which texture is used — the silhouette of a Diamond sphere and a Cobblestone sphere of the same diameter is identical. The texture decision is a *purely aesthetic* layer applied after the geometric algorithm has finished.

21.2 Block families and tones

For the survival builder choosing among the dozens of available blocks, the relevant question is not the texture index but the *tone family*: light/dark, warm/cool, uniform/patterned. The table below classifies the most common blocks used in Block Round builds:

Block	Family	Tone	Typical use
cobblestone	stone	gray	rustic walls, dungeons
stone	stone	gray	clean walls, plinths
oak_planks	wood	tan	floors, interior trim
quartz_block	quartz	white	temples, modern domes
sandstone	sand	tan	deserts, pyramids
deepslate	stone	dark	underground, accents

21.3 Gradients by mixing blocks

Although a single block type provides one tone, *gradients* can be obtained by interleaving two or three block types per layer. A classic Minecraft technique is the *stone-to-quartz* gradient on a dome: the bottom layers are Smooth Stone, the equatorial layers are a noisy mix of Smooth Stone and Diorite, and the top layers are Quartz. Each layer remains computed by the same Block Round algorithm; what changes is the per-voxel texture lookup. Block Round’s *Random* block option implements one such procedural mix by default (grass cap, dirt sub-layer, sparse ores in the middle, deepslate near bedrock). The mathematics is the same as Section 19’s layer decomposition; only the per-voxel “block tag” differs.

22. Conclusion

This document described, in a systematic way, the mathematical foundations employed in Block Round. In approximately one thousand lines of JavaScript code plus the supporting texture and audio pipelines, the project integrates contributions from several theoretical areas:

- **Analytic geometry:** implicit equations of the ellipse and ellipsoid as the primary criterion of belonging.
- **Classical rasterization algorithms:** Bresenham in the midpoint formulation, with the Pitteway/Kappel extension for ellipses.
- **Digital topology:** discrete boundary operators, concepts of 4-, 6- and 26-connectivity, slice composition and exposed-face counting.
- **Integral calculus:** volume of the ellipsoid as a triple integral and its discrete counterpart (counting measure over voxels).
- **Linear algebra:** cutting planes defined by linear inequalities and three-dimensional perspective projection.

- **Trigonometry**: parametrization of the camera in spherical coordinates and automatic distance adjustment as a function of FOV.
- **Combinatorial inventory arithmetic**: ceiling-division accounting of packs of 64 and double chests of 3 456 items, with the stacks-and-remainder representation displayed by the info chip.
- **Group symmetry**: exploitation of the octahedral group O_h and its order-8 reflection subgroup $\mathbb{Z}_2^3$ to reduce construction cost by a factor of 8 via `/clone`.

The central observation that the study allows to formulate is the following: on a discrete grid, **there is no single correct representation** of a continuous curve. Each algorithm presented in this document corresponds to a distinct mathematical definition of the cell-inclusion criterion, and each definition produces a silhouette with its own formal properties — smoothness in the Euclidean algorithm, the stepped pattern of Bresenham, larger coverage in the corner-coverage criterion. The choice of algorithm is therefore a technical decision that must consider the intended visual representation, the desired metric characteristics of the resulting shape, and — specifically for Block Round — the inventory and time cost of physically constructing the shape in-game.

Block Round occupies a small but mathematically rich slice of the Minecraft tooling ecosystem: it sits between purely geometric tools (WorldEdit, Litematica) and purely visual tools (resource packs, shaders), offering an explicit, derivable, and reproducible bridge from the continuous equation to the discrete placement plan. The companion project Pixel Round provides the same algorithms without the Minecraft layer, suitable for traditional pixel art and voxel art applications. Together the two projects exemplify how the same continuous-to-discrete mathematical pipeline supports radically different artistic and constructive workflows.

Appendix A: Construction reference tables and worked example

This appendix collects the most-used numerical and command references for Block Round users. The data below is consolidated from the per-section tables and worked examples scattered throughout this document, organised so the appendix functions as a one-page lookup for build planning, command syntax and an end-to-end worked example.

A.1 Comprehensive sphere reference (filled, hollow, packs)

The table below extends the canonical sphere tables of Sections 10 and 17 by adding the hollow-shell variant (*thin* mode) alongside the filled volume. The hollow-shell counts assume a single-voxel shell and are computed under the Euclidean algorithm; the thick mode adds roughly 10–15% more blocks to fill the empty diagonal cells as described in Section 12.

D	Filled (V)	Hollow (shell)	Packs filled	Packs hollow
8	268	170	5	3
10	523	316	9	5
12	904	500	15	8
14	1437	744	23	12
16	2145	1042	34	17
18	3052	1338	48	21
20	4189	1648	66	26
24	7238	2400	114	38
28	11494	3296	180	52
32	17156	4332	269	68

For a builder planning a survival run, the *hollow* column is usually the relevant one: only the visible shell needs the dedicated textured block, while the interior (if any is desired at all) can be filled with the cheapest material. The packs-filled column tells the total stockpile size for a fully filled sphere; packs-hollow is the realistic minimum for a hollow dome or shell.

A.2 WorldEdit / vanilla command reference

The following table summarises the WorldEdit and vanilla-Minecraft commands most relevant to constructing Block Round outputs in-game. WorldEdit commands assume the player holds a wand (default wooden axe) and has selected

a region; vanilla commands work without any mod but require coordinate arguments.

Command	Mod / origin	Result
<code>//sphere stone 8</code>	WorldEdit	filled sphere $r=8$ (Euclidean)
<code>//hsphere stone 8</code>	WorldEdit	hollow sphere $r=8$ (shell only)
<code>//cyl stone 8 16</code>	WorldEdit	cylinder $r=8$, $h=16$
<code>//ellipsoid st. 10 5 10</code>	WorldEdit	ellipsoid $10 \times 5 \times 10$
<code>/fill ... stone</code>	vanilla	fill box with one block type
<code>/clone ... mode:masked</code>	vanilla	copy region to new location

For Litematica users the workflow is different: rather than running commands, the schematic file (Block Round exports `.schem`, Sponge format v2) is loaded into the Litematica mod, which then displays a translucent ghost of the figure that the player places block-by-block. This is the closest survival-mode equivalent of the WorldEdit creative experience.

A.3 Worked example: a $D=20$ dome of Quartz on a temple

As a complete worked example, suppose a builder wants a Quartz dome of $D = 20$ on top of an existing stone temple. The temple top is 11×11 blocks, centered at $(0, 80, 0)$ in world coordinates. The dome is the upper half of a sphere of radius $r = 10$ cut at the $Y = 0$ plane. The construction plan is:

1. **Plan with Block Round.** Open the app, set 3D mode, Sphere, size $D = 20$, cut axis Y at 50%. Select Quartz Block from the picker. The info chip reports $V_{\text{dome}} \approx 2\,126$ blocks (≈ 34 packs, 1 double chest with 1 330 items remaining in the second).
2. **Gather resources.** $34 \text{ packs of Quartz Block} = \lceil 2\,126/4 \rceil = 532 \text{ Quartz Blocks} \times 4 \text{ Nether Quartz each} = 2\,128 \text{ Nether Quartz}$. Smelt or trade as appropriate.
3. **Decide algorithm.** For a Quartz dome viewed from below by passersby, the Euclidean algorithm gives the smoothest silhouette; for a dome on a remote mountain the Threshold algorithm reads cleaner. Block Round defaults to Euclidean.
4. **Position the dome.** The dome's flat face must coincide with the temple top. The dome's bounding box is $20 \times 10 \times 20$ blocks; place its centre at $(0, 80, 0)$, so the flat (cut) face lies at $y = 80$ and the dome apex is at $y = 89$.

5. **Build by symmetry.** Build the $+x, +z$ quadrant of the dome (about ≈ 530 blocks). Issue `/clone 0 80 0 10 89 10 -10 80 0 mode:masked` to mirror across x ; then `/clone -10 80 0 10 89 10 0 80 -10 mode:masked` to mirror across z . Total elapsed: ~ 10 minutes in survival.
6. **Polish.** Add Sea Lanterns at the apex and at the four cardinal points of the equator for emissive illumination (Block Round renders these with their emissive map at MC light level 15). Optionally toggle Block Round's edge overlay (key `g`) to verify the dome shell has no gaps before placing the last few blocks.

The mathematics of every step is exactly what the preceding twenty-two sections describe: implicit ellipsoid equation, Euclidean voxelisation, Y-axis cut, octahedral symmetry reduction via `/clone`, inventory arithmetic to translate the volume into packs and chests. The appendix is therefore not a new theory — it is a reminder that every theoretical concept of this document maps onto a single, concrete decision in the build-planning workflow.

A.4 Keyboard shortcut reference

The Block Round app exposes a small set of keyboard shortcuts that mirror the corner-button toggles on the canvas. They are listed below for quick reference; the shortcuts are global (no input field needs focus) and work in both 2D and 3D modes unless noted.

Key	Toggles	Notes
<code>g</code>	Edge grid overlay	black outline on each voxel face
<code>c</code>	Center guides	translucent yellow cross at axes
<code>d</code>	Download PNG	current canvas as PNG image
<code>i</code>	Info chip	block count and pack breakdown
<code>m</code>	2D / 3D toggle	switches between flat and voxel mode
<code>s</code>	Sound on/off	ambient block-place sounds
<code>t</code>	Day / night theme	dims background, keeps tiles readable
<code>Ctrl+Z</code> / <code>y</code>	Undo / redo	walks the figure-change history

The shortcuts `g` (grid) and `c` (center guides) are particularly useful during construction validation: `g` shows whether every voxel of the shell is in place (a missing block breaks the overlay pattern), and `c` confirms that the cuts are properly centred on the figure axes.

For the `Ctrl+Z` undo: Block Round walks every figure-changing edit (sliders, render mode, algorithm, shape, axis, block, mode toggle) but deliberately excludes visual-

only toggles (camera, edges overlay, center cross, theme, sound). This separation keeps the undo stack populated with edits the user is likely to want to reverse, not casual visual adjustments.

A.5 Glossary of key terms

Quick reference for the technical terms that recur throughout this document. Each entry briefly recapitulates the concept and points to the section where it is developed in detail.

- **Voxel** — unit cube in 3D, analogue of the pixel in 2D. Each Minecraft block is one voxel. See Section 2.
- **Pack (stack)** — in Minecraft, a stack of 64 identical items occupying one inventory slot. See Section 17.
- **Double chest** — two adjacent chests merged into a 54-slot container. Holds up to 3 456 items. See Section 17.
- **Shulker box** — portable 27-slot container that can itself be stored inside a chest slot. See Section 17.
- **Shell** — the outer one-voxel-thick layer of a solid figure. The relevant surface for survival builders. See Section 7 and 18.
- **Octant** — one of the eight regions of 3D space obtained by intersecting the three coordinate half-spaces. Used in symmetry optimisation. See Section 20.
- **UV mapping** — a 2D parametrisation of a 3D face, used to map a texture image onto each face of a voxel. See Section 21.
- **Bresenham** — 1965 incremental rasterisation algorithm using only integer arithmetic. See Section 5.
- **Threshold** — rasterisation criterion that includes a cell if any of its corners lies inside the ellipse. See Section 6.
- **Euclidean** — rasterisation criterion that includes a cell if its center lies inside the ellipse. See Section 4.
- **Easter egg** — a hidden feature triggered by a specific input. In Block Round: the oak tree and the creeper, both on slider value 15.
- **Schematic** — a serialised structure file (Sponge Schematic v2, `.schem`) loadable by WorldEdit, Litematica or MCEdit.

This glossary intentionally remains shorter than a full mathematical lexicon. Readers seeking deeper coverage of the underlying mathematics (digital topology, integral calculus, group theory, etc.) are referred to the relevant sections of this document.

A.6 Outlook and limits of the model

The Block Round model presented in this document covers the static voxelisation of ellipses, ellipsoids and their cuts plus the practical considerations introduced by the Minecraft layer. It does not address dynamic problems — moving figures, animated voxel transitions, fluid-simulation interactions — because those require a continuous-time framework that lies outside the discrete-rasterisation scope. A natural extension is the *schematic-level animation*: a sequence of static voxelisations connected by interpolation, which Block Round can already export as a `.schem` stream readable by Litematica scripts.

Another open direction is the *exact volume-preserving voxelisation*: the current discrete count V_{disc} converges to the continuous volume V_{cont} only asymptotically. For applications where the exact count must match the continuous formula (e.g. rendering competitions, mathematical art with strict block budgets), an extra refinement layer is required — for instance, by adjusting the radius slightly so the discrete count hits a target value. Block Round does not currently implement this refinement, but the underlying mathematics is straightforward: solve $V_{\text{disc}}(r) = V_{\text{target}}$ numerically for r .

Finally, the *higher-symmetry exploitation*: the present document covers the order-8 reflection subgroup of O_h , but the full order-48 group includes rotations that would in principle allow a $\times 48$ speedup. The practical obstacle is that the in-game commands work in axis-aligned regions, and exploiting the rotational symmetries requires a 45° -rotated build area. Server-side mods that internally rotate selection boxes (e.g. FAWE) can in principle achieve this, but mainstream tooling does not yet expose it. A reference implementation that demonstrates $\times 48$ speedup on a vanilla server would be a natural follow-up to the present work.

A.7 Further references and external resources

The mathematical concepts developed in this document have well-established literatures of their own. For readers seeking primary sources or deeper algorithmic background, the following entry points are recommended:

- J. E. Bresenham, *Algorithm for computer control of a digital plotter*, IBM Systems Journal, vol. 4 no. 1, 1965 — the original integer-arithmetic rasterisation

paper extended in this document.

- M. L. V. Pitteway, *Algorithm for drawing ellipses or hyperbolae with a digital plotter*, Computer Journal, vol. 10, 1967 — generalises Bresenham to arbitrary conics.
- A. Kappel, *An ellipse-drawing algorithm for raster displays*, ACM Comm. vol. 28, 1985 — the two-region elliptic formulation used in Section 5.
- R. Klette and A. Rosenfeld, *Digital Geometry: Geometric Methods for Digital Picture Analysis*, Morgan Kaufmann, 2004 — the standard reference for digital topology and the boundary operators of Section 7 and 18.
- WorldEdit documentation, enginehub.org/worldedit/ — official command reference for the `//sphere`, `//hsphere`, `//ellipsoid`, `//copy` family used throughout this document.
- Sponge Schematic specification v2, github.com/SpongePowered/Schematic-Specification — the file format Block Round exports as `.schem`.

The eight theoretical areas listed in Section 22 each have textbook-level treatments that go well beyond what a single technical document can summarise. Block Round’s contribution is not in the theory itself but in the specific synthesis: applying classical rasterisation to the textured-voxel Minecraft setting, with the explicit inventory and symmetry-cost considerations that are usually omitted from purely geometric tools.

A final remark on reproducibility: the entire pipeline of this document — the build script, the translations, the LaTeX template, the per-locale tables — is contained in the project repository’s `docs_math/` directory. Re-running `python build.py` on a system with XeLaTeX/MiKTeX produces this PDF exactly, in any of the nine supported locales. The companion project Pixel Round follows the same structure, allowing direct side-by-side comparison of the textured and untextured variants.

End of document

Block Round is maintained by Vinícius Rodrigues de Souza. The companion project Pixel Round is available at github.com/ViniSouza128/pixel-round; the current Block Round repository is at github.com/ViniSouza128/block-round.

Comments, corrections, and translation improvements are welcome via the issue tracker on the project repository. Native-speaker proofreading of the eight non-English locales is particularly valued, as the localisations were prepared with

substantial review but technical-mathematical terminology benefits from native correction.